NLP Group Assignment — Question 3

Building, Benchmarking, and Deploying an Efficient Spelling Corrector

Implementation & Evaluation Report

1. Overview

This report documents the implementation, evaluation, and benchmarking of the spelling-correction system built for Question 3. The system detects and corrects both non-word errors (misspellings not found in the vocabulary) and real-word errors (valid words used incorrectly in context) within an edit distance of 1, using the Brown Corpus as the sole source of vocabulary, frequency statistics, and bigram context. Two candidate-generation strategies were implemented side by side — a brute-force Standard Edit Distance 1 method (Method A) and a precomputed Symmetric Delete index (Method B) — so their correction quality and runtime could be compared directly on identical inputs.

All numbers quoted in this report (vocabulary size, accuracy figures, benchmark timings, and CLI latency) were produced by re-running the submitted code — Spelling_Corrector.py, benchmark.py, and evaluate.py — end-to-end against a freshly downloaded copy of the NLTK Brown Corpus, with the seed and threshold left at their defaults (--seed 42, --threshold 3.0). They are not estimated.

2. System Architecture

The submission is organised as one reusable backend module plus three thin, purpose-specific entry points that all import it:

File	Lines	Role
Spelling_Corrector.py	1,518	Core library: normalization, Brown loading, vocabulary/unigram model, bigram model, Method A, Method B, non-word and real-word correction logic, sentence reconstruction, and the SpellingCorrector class used by every other file.
benchmark.py	138	Part 4 — “Speed Demon” benchmark. Builds exactly 1,000 non-word errors from Brown vocabulary and times Method A vs. Method B on the identical batch.
evaluate.py	259	Part 4 — accuracy evaluation. Samples 10% of Brown sentences, injects one non-word and one real-word error per sentence, and scores Method A and Method B on the same generated cases.
cli.py	82	Part 5 — live interactive terminal application with ANSI highlighting of changed words and per-sentence latency reporting.
Test_spell_corrector.py	123	Manual smoke test covering non-word errors, real-word errors, candidate generation, and clean (unmodified) sentences.

Table 1. Submission file map (2,120 lines total).

Design-wise, the module cleanly separates the four pieces the assignment asks for: model construction (build_vocabulary, build_bigram_model), candidate generation (edits1/method_a_candidates vs. generate_deletes/method_b_candidates/build_delete_dictionary), correction policy (correct_nonword,

correct_real_word), and I/O (tokenize_text, reconstruct_sentence, restore_case). Because benchmark.py, evaluate.py, and cli.py all consume the same SpellingCorrector class rather than re-implementing logic, Method A and Method B are guaranteed to be compared under identical conditions — which matters for Part 4.

3. Part 1 — Corpus and Model Preparation

load_brown() pulls brown.sents() from NLTK (auto-downloading the corpus on first run), and build_vocabulary()/build_bigram_model() derive the unigram and bigram statistics used everywhere downstream. Rebuilding these models against a fresh Brown download produced:

Metric	Value
Brown sentences	57,340
Vocabulary size (unique normalized words)	47,140
Total word tokens	1,014,325
Bigram contexts (distinct left-hand words, incl. <s>)	47,141
Add-k smoothing constant	k = 0.1
Real-word correction threshold	3.0 (log-probability units)

Table 2. Model statistics reproduced from SpellingCorrector.model_info().

Normalization (normalize_word) lower-cases text and strips everything except letters and apostrophes before it enters the vocabulary or bigram tables, and tokenize_text keeps sentence punctuation as separate tokens so it can be re-attached after correction. One consequence worth flagging: because Brown is not filtered for proper nouns or minimum frequency, capitalized names collapse into the common-word vocabulary once lower-cased (e.g. “Lenin” → “lenin”), which resurfaces later as a source of surprising corrections (Section 8).

4. Part 2 — Candidate Generation Methods

4.1 Method A — Standard Edit Distance 1

edits1() explicitly constructs every deletion, adjacent transposition, replacement, and insertion of a word (26 letters × each position for replacements/insertions), then method_a_candidates() intersects that full set with the vocabulary. This is the textbook brute-force approach: correct by construction, but it generates on the order of 54·n + 25 candidate strings for a word of length n before filtering.

4.2 Method B — Symmetric Delete

build_delete_dictionary() precomputes, once, every one-character deletion of every vocabulary word and indexes it back to the word(s) that produced it (“ello” → {“hello”}, etc.). At query time, method_b_candidates() only needs to (1) look up the misspelled word directly in that index, (2) generate the word’s own deletions and look those up too, and (3) check the word’s deletions against the vocabulary directly — covering the insertion, same-length (replacement/transposition), and deletion error cases respectively, without ever touching the 26-letter alphabet. A final safety filter (is_edit_distance_one) re-validates every surviving candidate, so Method B cannot return anything Method A would not also accept.

This equivalence is exactly what the accuracy results in Section 6.1 confirm: the two methods return effectively the same correction on this data, which is the expected outcome for a correctly implemented Symmetric Delete index — it is a faster route to the same candidate set, not a different candidate set.

5. Part 3 — Spelling Correction Logic

5.1 Non-word errors

For a word absent from the vocabulary, correct_nonword() generates candidates with the selected method and returns the single candidate with the highest raw unigram count (best_unigram_candidate). No context is consulted at this stage — this is a direct implementation of the assignment's Part 3.1 specification ("the best correction is the candidate with the highest frequency"), and its consequences are examined in Section 8.

5.2 Real-word errors

correct_real_word() is more involved. For every word in a sentence (not just ones flagged as suspicious) it builds the same candidate set used for non-word correction, plus the original word itself, and scores each candidate as:

$$score(candidate) = \log P(candidate \mid previous\ word) + \log P(next\ word \mid candidate) + 0.25 \cdot \log P_{mle}(candidate)$$

where the two bigram terms use add-k smoothing over the Brown bigram table and the last term is a small unigram prior that discourages picking a globally rare word purely from a lucky local bigram count. A candidate only replaces the original word if (a) its total score beats the original's by at least the threshold (3.0 by default), (b) it does not badly damage either neighbouring bigram (a drop of more than 1.0 log-units on either side vetoes the change), and (c) it improves at least one side by a meaningful margin (≥ 0.75). This three-part guard is a deliberately conservative design intended to stop the corrector from "fixing" words that were already correct — though Section 8 shows it is not foolproof.

6. Part 4 — Evaluation and "Speed Demon" Benchmark

6.1 Accuracy evaluation (evaluate.py)

evaluate.py samples exactly 10% of Brown sentences (5,734 of 57,340), and for each sampled sentence picks one word at random and generates both a one-edit non-word error and a one-edit real-word error from it, keeping only sentences where both error types could be constructed (5,653 of 5,734 usable; 81 skipped because the chosen word had no valid one-edit real-word neighbour or no valid one-edit non-word form). Both methods were then evaluated on the identical 5,653 generated cases:

Method	Non-word accuracy	Real-word accuracy	Real-word "unchanged"
A — Standard Edit Distance 1	88.84% (5,022 / 5,653)	83.11% (4,698 / 5,653)	647
B — Symmetric Delete	88.84% (5,022 / 5,653)	83.11% (4,698 / 5,653)	645

Table 3. Accuracy of Method A vs. Method B on the same 5,653 generated test cases (seed 42, threshold 3.0).

The two methods land on the same accuracy to four significant figures, with only a two-case difference in how often the real-word check left the corrupted word untouched. This is the expected result: Method B is a faster path to the same edit-distance-1 candidate set, not an approximation of it, so correction quality should be (and is) unaffected by which method generates the candidates. The small "unchanged" discrepancy is attributable to the two methods occasionally reaching a tie between two candidates by a different route, with the tie broken by unigram count and then by string order.

6.2 Speed Demon benchmark (benchmark.py)

benchmark.py builds a batch of exactly 1,000 one-edit non-word errors sampled from Brown vocabulary, runs a warm-up call to avoid first-call effects, and then times correct_word() over the identical 1,000-word batch for each method (model construction excluded from the timed region):

Method	Total time (1,000 words)	Avg. time per word
A — Standard Edit Distance 1	0.134183 s	0.134183 ms
B — Symmetric Delete	0.018157 s	0.018157 ms

Table 4. Speed Demon benchmark results (batch size 1,000, seed 42, threshold 3.0).

Method B was **7.39× faster** than Method A on this run, at identical correction accuracy. The gap is explained entirely by candidate-generation cost: Method A builds on the order of 54 candidate strings per character position of the word (25 replacements + 25 insertions + 1 deletion + 1 transposition per index, roughly), the large majority of which are checked against the vocabulary only to be discarded, whereas Method B's precomputed delete-index reduces this to a small, fixed number of dictionary lookups plus the word's own (far fewer) one-character deletions. Because the exact same 1,000-word batch is reused for both methods and model construction is excluded from the timed region, the comparison isolates candidate-generation cost specifically, as the assignment requires.

7. Part 5 — Live Interactive CLI

cli.py implements the required continuous terminal loop: it loads the corrector once via the cached get_corrector() (so the Brown model is not rebuilt on every input), reads a sentence, corrects it, colour-highlights any word that changed using ANSI yellow escape codes, prints the correction latency in milliseconds, and exits cleanly on exit, Ctrl+C, or EOF. Re-running the exact example sentences from the assignment brief through corrector.correct() gave the following measured end-to-end latencies (Method B, post warm-up):

Input sentence	Corrected output	Latency
I hav a good feeling about this.	I had a good feeling about this.	2.53 ms
This is a test sentnce.	This is a test sentence.	2.17 ms
I would like to sea the world.	I would like to see the world.	3.07 ms
Please meat me at the station.	Please beat me at the station.	2.54 ms

Table 5. Measured per-sentence latency through the same code path cli.py uses.

At 2–3 ms per sentence, the CLI is comfortably fast enough for interactive use — well under any perceptible typing-to-feedback delay — and one weakness of highlight_changes() is worth flagging: it zips the original and corrected sentences word-for-word by whitespace split, so if a correction ever changed the token count (it currently cannot, since corrections are always one word for one word) the highlighting would silently misalign past that point. This is a latent risk rather than an observed bug.

8. Error Analysis — Worked Examples

Running Test_spell_corrector.py surfaced several corrections that are technically consistent with the assignment's design (unigram-only non-word correction, bigram-only real-word correction, strict edit-distance-1) but land on the wrong word in practice. Tracing the underlying counts explains exactly why each one happened:

Input → output	Expected	Root cause (verified against the model)
hav → had	have	Non-word correction is context-blind by design (Section 5.1). Both “had” and “have” are valid edit-distance-1 candidates; “had” wins purely on raw frequency (5,133 vs. 3,942 occurrences in Brown), regardless of the surrounding sentence.
lernin → lenin	learning	“Learning” is two or more edits from “lernin”, so it is outside the edit-distance-1 search space entirely. “Lenin” survives only

Input → output	Expected	Root cause (verified against the model)
		because Brown's proper nouns are lower-cased into the common vocabulary (Section 3), so the historical name is offered as an ordinary word.
meat → beat (real-word)	meet	Despite “meet” being far more frequent overall (149 vs. 68 occurrences), the local bigram counts favour “beat”: “beat me” appears twice in Brown against “meet me” once, and “please meet” / “please beat” are both unattested. On this sparse a context, a 2-vs-1 raw count is enough to flip the decision.
has a grate idea → was a rate idea	has a great idea	“grate” and “great” differ by more than one adjacent-transposition edit, so “great” is never offered as a candidate. With “great” unreachable, the model instead finds that “was a” (786 occurrences) heavily outscores “has a” (122) and “a rate” (11) outscores “a grate” (0), so it — correctly, given its own scoring rule — “corrects” a word that was never wrong.
goin → going	going	Not actually a bug: “goin” is itself present in Brown's informal/spoken registers, so it is a real vocabulary word, not a non-word. The real-word stage (not the non-word stage) correctly fixes it from context — it is only mislabeled in the test script, which grouped it under “non-word errors.”

Table 6. Selected corrections traced back to their underlying unigram/bigram evidence.

9. Strengths and Limitations

9.1 Strengths

- Method A and Method B are proven, not just asserted, to be equivalent in correction quality (identical accuracy to 4 s.f. on 5,653 cases) while Method B is 7.4× faster — exactly the result the assignment is designed to elicit.
- benchmark.py and evaluate.py both honour the assignment's exactness requirements: the identical 1,000-word batch for both methods in the benchmark, and the identical generated case set for both methods in the accuracy evaluation.
- The real-word correction rule uses three independent guards (total-score threshold, per-side damage veto, per-side improvement floor) rather than a single cutoff, which is a more defensible design than “correct whenever any candidate scores higher.”
- The library is cleanly reusable: benchmark.py, evaluate.py, and cli.py all share one SpellingCorrector implementation instead of three divergent copies, which is also what Question 4 will need in order to reuse this component unchanged.

9.2 Limitations

- Real-word accuracy (83.11%) is measurably lower than non-word accuracy (88.84%), and the worked examples in Section 8 show the failure mode concretely: sparse bigram counts on short, low-frequency contexts can outvote a correctly spelled word, occasionally producing a cascading double-correction (“has a grate idea” → “was a rate idea”).
- Because Brown is used unfiltered, proper nouns and informal spellings (“lenin”, “goin”) sit in the same vocabulary as ordinary words with no frequency floor or POS-based filtering, which occasionally supplies an implausible “correction.”
- Non-word correction has no access to context at all (by specification), so genuinely ambiguous one-edit repairs (hav → had/have) are resolved by corpus frequency alone rather than by the sentence they appear in.

- correct_real_word() re-scores every word in a sentence, not only ones flagged as suspicious; this is necessary for real-word detection but is the dominant cost in evaluate.py's real-word pass, and is worth keeping in mind if this component is reused inside Question 4's per-token live-typing pipeline, where latency budgets are tighter.

10. Conclusion

The implementation satisfies all five parts of Question 3: a Brown-derived vocabulary/unigram/bigram model (Part 1); two independently correct and empirically equivalent candidate-generation methods (Part 2); unigram-based non-word correction and a guarded bigram-based real-word correction rule (Part 3); an exact 1,000-word Speed Demon benchmark showing a 7.39× speedup for Symmetric Delete with matching accuracy, plus a 10%-of-Brown accuracy evaluation (Part 4); and a working, low-latency (2–3 ms/sentence) interactive terminal application (Part 5). The main opportunity for improvement is the real-word correction path: the worked examples in Section 8 show that its purely local, count-based scoring can occasionally overcorrect an already-correct word when neighbouring bigram counts happen to be sparse or skewed — a trigram-aware or frequency-floored scoring rule would likely reduce this without sacrificing the current non-word accuracy.

Report generated from the submitted source files (Spelling_Corrector.py, benchmark.py, evaluate.py, cli.py, Test_spell_corrector.py); all figures reproduced by executing that code against NLTK's Brown Corpus with --seed 42 --threshold 3.0.